

조건문

1. Boolean 자료형과 if 조건문

시작하기 전에

- Boolean

- 불린 / 불리언 / 불
- True와 False 값만 가질 수 있음

```
>>> print(True)
True
>>> print(False)
False
```

불 만들기 : 비교 연산자

- 불은 비교 연산자를 통해 생성, 파이썬에는 여섯 개의 비교 연산자가 있음

연산자	설명	연산자	설명
==	같다	>	크다
!=	다르다	<=	작거나 같다
<	작다	>=	크거나 같다

불 만들기 : 비교 연산자

- 숫자 또는 문자열에 적용

```
>>> print(10 == 100)
False
>>> print(10 != 100)
True
>>> print(10 < 100)
True
>>> print(10 > 100)
False
>>> print(10 <= 100)
True
>>> print(10 >= 100)
False
```

조건식	의미	결과
10 == 100	10과 100은 같다	거짓
10 != 100	10과 100은 다르다	참
10 < 100	10은 100보다 작다	참
10 > 100	10은 100보다 크다	거짓
10 <= 100	10은 100보다 작거나 같다	참
10 >= 100	10은 100보다 크거나 같다	거짓

불 만들기 : 비교 연산자

- 문자열에도 비교 연산자 적용 가능
 - 한글은 사전 순서(가나다순)로 앞에 있는 것이 작은 값

```
>>> print("가방" == "가방")
True
>>> print("가방" != "하마")
True
>>> print("가방" < "하마")
True
>>> print("가방" > "하마")
False
```

불 연산하기 : 논리 연산자

- 불끼리 논리 연산자 사용 가능

연산자	의미	설명
not	아니다	불을 반대로 전환합니다.
and	그리고	피연산자 두 개가 모두 참일 때 True를 출력하며, 그 외는 모두 False를 출력합니다.
or	또는	피연산자 두 개 중에 하나만 참이라도 True를 출력하며, 두 개가 모두 거짓일 때만 False를 출력합니다.

● not 연산자

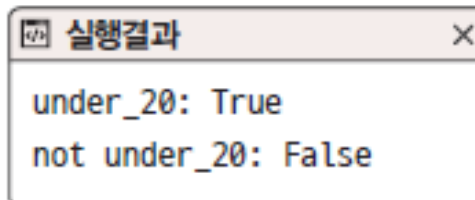
- 단항 연산자
- 참과 거짓 반대로 바꿈

```
>>> print(not True)
False
>>> print(not False)
True
```

불 연산하기 : 논리 연산자

- 예시 – not 연산자 조합하기

```
01 x = 10
02 under_20 = x < 20
03 print("under_20:", under_20)
04 print("not under_20:", not under_20)
```



실행결과

under_20: True
not under_20: False

불 연산하기 : 논리 연산자

- and 연산자와 or 연산자

- and 연산자는 양쪽 변의 값이 모두 참일 때만 True를 결과로 냄

- and 연산자

좌변	우변	결과
True	True	True
True	False	False
False	True	False
False	False	False

- or 연산자

좌변	우변	결과
True	True	True
True	False	True
False	True	True
False	False	False

불 연산하기 : 논리 연산자

- 예시 – and 연산자와 or 연산자

"사과 그리고 배 가져와!"

"사과 또는 배 가져와!"

"치킨(True) 그리고 쓰레기(False) 가져와!"

"치킨(True) 또는 쓰레기(False) 가져와!"

```
>>> print(True and True)
True
>>> print(True and False)
False
>>> print(False and True)
False
>>> print(False and False)
False
>>> print(True or True)
True
>>> print(True or False)
True
>>> print(False or True)
True
>>> print(False or False)
False
```

논리 연산자의 활용

- and 연산자



- or 연산자



if 조건문이란

- if 조건문

- 조건에 따라 코드 실행하거나 실행하지 않게 할 때 사용하는 구문
- 조건 분기

if 불 값이 나오는 표현식: → if의 조건문 뒤에는 반드시 콜론(:)을 붙여줘야 합니다.

□□□ 불 값이 참일 때 실행할 문장

□□□ 불 값이 참일 때 실행할 문장

□□□□는 들여쓰기 4칸

↓
if문 다음 문장은 4칸 들여쓰기 후 입력합니다.

if 조건문이란

- 예시

```
>>> if True: Enter
    print("True입니다...!") Enter
    print("정말 True입니다...!") Enter
    Enter
True입니다...!
정말 True입니다...!
```

```
>>> if False: Enter
    print("False입니다...!") Enter
    Enter
>>>
```

if 조건문이란

- 예시 – 조건문의 기본 사용

```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  number = int(number)  
04  
05  # 양수 조건  
06  if number > 0:  
07      print("양수입니다")  
08  
09  # 음수 조건  
10  if number < 0:  
11      print("음수입니다")  
12  
13  # 0 조건  
14  if number == 0:  
15      print("0입니다")
```

실행결과 1

정수 입력> 273 Enter

양수입니다

실행결과 2

정수 입력> -52 Enter

음수입니다

실행결과 3

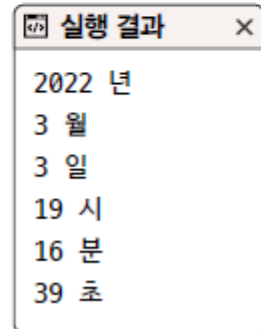
정수 입력> 0 Enter

0입니다

날짜/시간 활용하기

- 예시 – 날짜/시간 출력하기
 - `datetime.datetime.now()` 함수

```
01  # 날짜/시간과 관련된 기능을 가져옵니다.
02  import datetime
03
04  # 현재 날짜/시간을 구합니다.
05  now = datetime.datetime.now()
06
07  # 출력합니다.
08  print(now.year, "년")
09  print(now.month, "월")
10  print(now.day, "일")
11  print(now.hour, "시")
12  print(now.minute, "분")
13  print(now.second, "초")
```



실행 결과

```
2022 년
3 월
3 일
19 시
16 분
39 초
```

날짜/시간 활용하기

- 예시 – 날짜/시간을 한 줄로 출력하기

```
01  # 날짜/시간과 관련된 기능을 가져옵니다.  
02  import datetime  
03  
04  # 현재 날짜/시간을 구합니다.  
05  now = datetime.datetime.now()  
06  
07  # 출력합니다.  
08  print("{}년 {}월 {}일 {}시 {}분 {}초".format(  
09      now.year,  
10      now.month,  
11      now.day,  
12      now.hour,  
13      now.minute,  
14      now.second  
15  ))
```

실행 결과

2022년 3월 3일 19시 18분 45초

날짜/시간 활용하기

- 예시 – 오전과 오후를 구분하는 프로그램

```
01  # 날짜/시간과 관련된 기능을 가져옵니다.
02  import datetime
03
04  # 현재 날짜/시간을 구합니다.
05  now = datetime.datetime.now()
06
07  # 오전 구분
08  if now.hour < 12:
09      print("현재 시각은 { }시로 오전입니다!".format(now.hour))
10
11  # 오후 구분
12  if now.hour >= 12:
13      print("현재 시각은 { }시로 오후입니다!".format(now.hour))
```

실행결과

현재 시각은 19시로 오후입니다!

날짜/시간 활용하기

- 예시 – 계절을 구분하는 프로그램

```
01  # 날짜/시간과 관련된 기능을 가져옵니다.
02  import datetime
03
04  # 현재 날짜/시간을 구합니다.
05  now = datetime.datetime.now()
06
07  # 봄 구분
08  if 3 <= now.month <= 5:
09      print("이번 달은 {}월로 봄입니다!".format(now.month))
10
11  # 여름 구분
12  if 6 <= now.month <= 8:
```

날짜/시간 활용하기

```
13     print("이번 달은 {}월로 여름입니다!".format(now.month))
14
15     # 가을 구분
16     if 9 <= now.month <= 11:
17         print("이번 달은 {}월로 가을입니다!".format(now.month))
18
19     # 겨울 구분
20     if now.month == 12 or 1 <= now.month <= 2:
21         print("이번 달은 {}월로 겨울입니다!".format(now.month))
```

 실행결과

×

이번 달은 3월로 봄입니다!

짝수와 홀수 구분하기(누적예제)

- 예시 - 끝자리로 짝수와 홀수 구분

```
01  # 입력을 받습니다.
02  number = input("정수 입력> ")
03
04  # 마지막 자리 숫자를 추출
05  last_character = number[-1]
06
07  # 숫자로 변환하기
08  last_number = int(last_character)
09
10  # 짝수 확인
11  if last_number == 0 \
12     or last_number == 2 \
13     or last_number == 4 \
14     or last_number == 6 \
15     or last_number == 8:
16     print("짝수입니다")
17
```

짝수와 홀수 구분하기(누적예제)

```
18  # 홀수 확인
19  if last_number == 1 \
20     or last_number == 3 \
21     or last_number == 5 \
22     or last_number == 7 \
23     or last_number == 9:
24     print("홀수입니다")
```

실행 결과 1

정수 입력> 52

짝수입니다

실행 결과 2

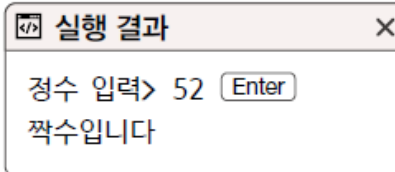
정수 입력> 273

홀수입니다

짝수와 홀수 구분하기(누적예제)

- 예시 - in 문자열 연산자를 활용해서 짝수와 홀수 구분

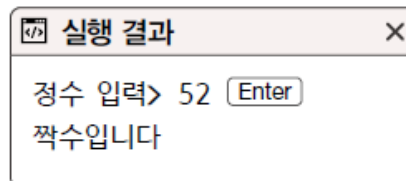
```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  last_character = number[-1]  
04  
05  # 짝수 조건  
06  if last_character in "02468":  
07      print("짝수입니다")  
08  
09  # 홀수 조건  
10  if last_character in "13579":  
11      print("홀수입니다")
```



짝수와 홀수 구분하기(누적예제)

- 예시 - 나머지 연산자를 활용해서 짝수와 홀수 구분

```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  number = int(number)  
04  
05  # 짝수 조건  
06  if number % 2 == 0:  
07      print("짝수입니다")  
08  
09  # 홀수 조건  
10  if number % 2 == 1:  
11      print("홀수입니다")
```



2. if~else와 elif 구문

시작하기 전에

- 정반대되는 상황에서 두 번이나 if 조건문을 사용해 조건을 비교하는 것은 낭비일 수 있다.

```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  number = int(number)  
04  
05  # 짝수 조건  
06  if number % 2 == 0:  
07      print("짝수입니다")  
08  
09  # 홀수 조건  
10  if number % 2 == 1:  
11      print("홀수입니다")
```

else 조건문의 활용

- else 구문

- if 조건문 뒤에 사용하며, if 조건문의 조건이 거짓일 때 실행되는 부분

```
if 조건:
```

```
    조건이 참일 때 실행할 문장
```

```
else:
```

```
    조건이 거짓일 때 실행할 문장
```

조건은 들여쓰기 4칸

- 조건문이 오로지 두 가지로만 구분될 때 if else 구문을 사용하면 조건 비교를 단 한번만 하므로 이전의 코드보다 두 배 효율적

else 조건문의 활용

- 예시 – if 조건문에 else 구문 추가해서 짝수와 홀수 구분

```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  number = int(number)  
04  
05  # 조건문을 사용합니다.  
06  if number % 2 == 0:  
07      # 조건이 참일 때, 즉 짝수 조건  
08      print("짝수입니다")  
09  else:  
10      # 조건이 거짓일 때, 즉 홀수 조건  
11      print("홀수입니다")
```



elif 구문

- elif 구문

- 세 개 이상의 조건을 연결해서 사용
- if 조건문과 else 구문 사이에 입력

```
if 조건A:  
    조건A가 참일 때 실행할 문장  
elif 조건B:  
    조건B가 참일 때 실행할 문장  
elif 조건C:  
    조건C가 참일 때 실행할 문장  
...  
else:  
    모든 조건이 거짓일 때 문장
```

□□□□는 들여쓰기 4칸

elif 구문

- 예시 – 계절 구하기

```
01  # 날짜/시간과 관련된 기능을 가져옵니다.
02  import datetime
03
04  # 현재 날짜/시간을 구하고
05  # 쉽게 사용할 수 있게 월을 변수에 저장합니다.
06  now = datetime.datetime.now()
07  month = now.month
08
09  # 조건문으로 계절을 확인합니다.
10  if 3 <= month <= 5:
11      print("현재는 봄입니다.")
12  elif 6 <= month <= 8:
13      print("현재는 여름입니다.")
14  elif 9 <= month <= 11:
15      print("현재는 가을입니다.")
16  else:
17      print("현재는 겨울입니다.")
```

실행결과 ×

현재는 봄입니다

if 조건문을 효율적으로 사용하기(누적 예제)

- 조건문의 활용

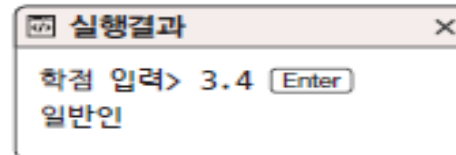
조건	설명(학생 평가)	조건	설명(학생 평가)
4.5	신	1.75~2.3	오락문화의 선구자
4.2~4.5	교수님의 사랑	1.0~1.75	불가촉천민
3.5~4.2	현 체제의 수호자	0.5~1.0	자벌레
2.8~3.5	일반인	0~0.5	플랑크톤
2.3~2.8	일탈을 꿈꾸는 소시민	0	시대를 앞서가는 혁명의 씨앗

- 예시 - 유머를 조건문으로 구현하기(1)

```
01  # 변수를 선언합니다.
02  score = float(input("학점 입력> "))
03
04  # 조건문을 적용합니다.
05  if score == 4.5:
06      print("신")
07  elif 4.2 <= score < 4.5:
08      print("교수님의 사랑")
```

if 조건문을 효율적으로 사용하기(누적 예제)

```
09 elif 3.5 <= score < 4.2:
10     print("현 체제의 수호자")
11 elif 2.8 <= score < 3.5:
12     print("일반인")
13 elif 2.3 <= score < 2.8:
14     print("일탈을 꿈꾸는 소시민")
15 elif 1.75 <= score < 2.3:
16     print("오락문화의 선구자")
17 elif 1.0 <= score < 1.75:
18     print("불가촉천민")
19 elif 0.5 <= score < 1.0:
20     print("자벌레")
21 elif 0 < score < 0.5:
22     print("플랑크톤")
23 elif score == 0:
24     print("시대를 앞서가는 혁명의 씨앗")
```



- 위에서 제외된 조건을 한 번 더 검사하여 비효율적

if 조건문을 효율적으로 사용하기(누적 예제)

- 예시 - 유머를 조건문으로 구현하기(2)

```
01 # 변수를 선언합니다.
02 score = float(input("학점 입력> "))
03
04 # 조건문을 적용합니다.
05 if score == 4.5:
06     print("신")
07 elif 4.2 <= score:
08     print("교수님의 사랑")
09 elif 3.5 <= score:
10     print("현 체제의 수호자")
11 elif 2.8 <= score:
12     print("일반인")
13 elif 2.3 <= score:
14     print("일탈을 꿈꾸는 소시민")
15 elif 1.75 <= score:
16     print("오락문화의 선구자")
17 elif 1.0 <= score:
18     print("불가촉천민")
19 elif 0.5 <= score:
20     print("자벌레")
21 elif 0 < score:
22     print("플랑크톤")
23 else:
24     print("시대를 앞서가는 혁명의 씨앗")
```

하위 값만 검사하고 상위 값은 검사를 생략

elif 4.2 <= score < 4.5:

→ elif 4.2 <= score:

조건 비교를 반으로 줄이고 코드 가독성 향상됨

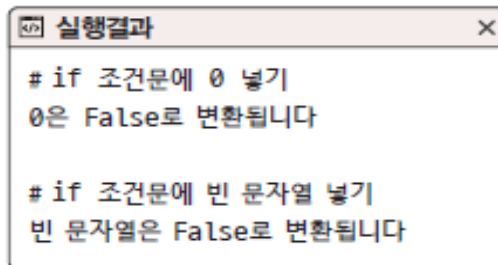


False로 변환되는 값

- 빈 컨테이너

- if 조건문의 매개변수에 볼 아닌 다른 값이 올 때 자동으로 볼로 변환
- 이 때 False로 변환되는 값: None, 0.0, 빈 문자열, 빈 바이트열, 빈 리스트
- 예시 - False로 변환되는 값

```
01 print("# if 조건문에 0 넣기")
02 if 0:
03     print("0은 True로 변환됩니다")
04 else:
05     print("0은 False로 변환됩니다")
06 print()
07
08 print("# if 조건문에 빈 문자열 넣기")
09 if "":
10     print("빈 문자열은 True로 변환됩니다")
11 else:
12     print("빈 문자열은 False로 변환됩니다")
```



```
실행결과
# if 조건문에 0 넣기
0은 False로 변환됩니다

# if 조건문에 빈 문자열 넣기
빈 문자열은 False로 변환됩니다
```

pass 키워드

- 나중에 구현하고자 구문을 비워 두는 경우

```
if zero == 0
    빈 줄 삽입
else:
    빈 줄 삽입
```

- 예시 - 나중에 구현하려고 비워 둔 구문

```
01  # 입력을 받습니다.
02  number = input("정수 입력> ")
03  number = int(number)
04
05  # 조건문 사용
06  if number > 0:
07      # 양수일 때: 아직 미구현 상태입니다.
08  else:
09      # 음수일 때: 아직 미구현 상태입니다.
```

실행 결과

```
File "pass_keyword.py", line 8
    else:
    ^
IndentationError: expected an indented block
```

pass 키워드

- IndentationError

- if 조건문 사이에는 무조건 들여쓰기 4칸 넣고 코드 작성해야 함

- pass 키워드 - 아무것도 작성하지 않고 임시적으로 비워 둠

- 예시 - pass 키워드를 사용한 미구현 부분 입력

```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  number = int(number)  
04  
05  # 조건문 사용  
06  if number > 0:  
07      # 양수일 때: 아직 미구현 상태입니다.  
08      pass  
09  else:  
10      # 음수일 때: 아직 미구현 상태입니다.  
11      pass
```

raise NotImplementedError

- raise NotImplementedError 를 사용하면 "아직 구현하지 않은 부분이에요!"라는 오류를 강제로 발생

```
01  # 입력을 받습니다.  
02  number = input("정수 입력> ")  
03  number = int(number)  
04  
05  # 조건문 사용  
06  if number > 0:  
07      # 양수일 때: 아직 미구현 상태입니다.  
08      raise NotImplementedError  
09  else:  
10      # 음수일 때: 아직 미구현 상태입니다.  
11      raise NotImplementedError
```

 오류

정수 입력> 10

Traceback (most recent call last):

File "passkeyword01.py", line 8, in <module>

raise NotImplementedError

NotImplementedError